TECHNICAL PROJECT & BENCHMARK ASSESSMENT REPORT

Multi-Outlet POS Platform Performance, Optimization & Scalability Assessment

Documenting technical investigation methodologies, multi-tenant transaction concurrency, database lock mitigation, and scalability modeling for high-throughput retail & food-service platforms.

Organization & Author:
LOGIXOTECH Technical Engineering Team

Industry Focus:
Retail, Quick-Service & Food Beverage (F&B)

Publication Date:
November 2026

Case Study Subject:
Perera & Sons POS Platform Engagement

Report Classification:
Verified Technical Project & Benchmark Assessment

Target Benchmark Context:
Multi-Location High-Concurrency Shared Database Architecture (~350 Outlets)

3. EXECUTIVE SUMMARY

Engagement Context and High-Level Objectives

Engagement Overview & Mandate

ENGINEERING MANDATE

This technical project and benchmark report documents LOGIXOTECH's technical engagement involving the **Perera & Sons multi-outlet Point of Sale (POS) platform**. Perera & Sons operates an extensive retail and food-service network requiring reliable, high-volume transaction processing across dozens of commercial outlets feeding into a centralized database backend.

Important Project Sequence: The Perera & Sons POS platform was an existing, active production system. Following the release and rollout of additional POS application features, operational outlets experienced noticeable performance degradation, elevated checkout response times, and intermittent transaction delays during peak meal service hours. LOGIXOTECH was specifically contracted to perform an independent technical investigation, identify root-cause database and API bottlenecks, and execute targeted performance optimizations.

Technical Scope: LOGIXOTECH did not develop the original POS codebase or implement the newly introduced features. Our team was assigned the critical engineering task of profiling query execution, analyzing lock contention, reviewing connection pool exhaustion, and re-engineering hot data paths to preserve transaction consistency while drastically reducing response latencies.

MULTI-LOCATION BUSINESS ENVIRONMENT CHARACTERISTICS

Operational Constraints Governing High-Throughput POS Networks

High-Frequency Storefront Operations

STOREFRONT

- **Centralized Processing:** Outlets process walk-in customer sales, takeaway packaging, and kitchen tickets against shared central database services.
- **Simultaneous Active Users:** Hundreds of concurrent floor cashiers, counter managers, and kitchen staff interacting with the backend in real time.
- **Strict Invariant Requirements:** Zero tolerance for missing sales records, duplicate ticket items, or out-of-sync retail pricing.

System Stress Under Peak Load

BOTTLENECKS

- **Synchronous Latency Cascades:** Slow queries during peak meal rushes directly blocked floor checkout terminals, leading to cashier UI freezes.
- **Connection Pool Saturation:** Surging terminal connection requests exhausted database worker allocations, causing connection timeouts.
- **Row Lock Contention:** Concurrent modifications to inventory and settlement tables triggered transaction serialization waits.

4. PROJECT BACKGROUND: CENTRALIZED ARCHITECTURE ROLE

Mission-Critical Functionality of the POS Backend

Sales & Settlement

CORE POS

Atomic processing of customer receipts, line items, payment splits, discounts, and tax calculations with guaranteed financial ledger consistency.

Catalog & Pricing

REFERENCE DATA

Central distribution of fast-changing item catalogs, regional price tiers, combo discounts, and promotional campaigns across all outlet terminals.

Operational Visibility

REPORTING

Real-time sales rollup dashboards, hourly velocity metrics, stock depletion alerts, and end-of-day reconciliation reports for management.

5. LOGIXOTECH TECHNICAL INVESTIGATION WORKSTREAMS

Systematic Deconstruction of Performance Degradation

A. Database Performance Investigation — POSTGRESQL ENGINE

LOGIXOTECH executed exhaustive telemetry and diagnostic profiling on the database cluster:

- **Slow Query Profiling:** Monitored query execution frequency, total elapsed execution time, and buffer cache misses using `pg_stat_statements`.
- **Execution Plan Deconstruction:** Ran `EXPLAIN ANALYZE` on critical sales recording and receipt queries, uncovering unindexed sequential table scans.
- **Resource Utilization Review:** Discovered database CPU spikes reaching 85%+ during meal peaks due to repeated sorting and in-memory hash joins.
- **Connection Starvation:** Identified unpooled client connections where each terminal spawned direct backend connections, hitting server connection caps.

B. API & Application Performance Review — DATA ACCESS LAYER

Investigating backend services and application-to-database communication patterns:

- **N+1 Query Cascades:** Discovered that order retrieval endpoints fetched parent order headers and then triggered individual queries for each line item.
- **Unnecessary Polling Loops:** Storefront terminals were executing unthrottled polling intervals every few seconds to refresh menu status, saturating bandwidth.
- **Over-Fetching Payloads:** Endpoints retrieved full relational entity graphs with dozens of unneeded columns, inflating JSON serialization times.
- **Synchronous Blocking Workflows:** Long-running inventory and receipt email tasks were executed within the primary request-response thread.

C. Transaction & Concurrency Investigation — LOCKING & CONTENTION

Analyzing transaction behavior under simultaneous multi-terminal checkout activity:

- **Row Lock Queues:** Multiple terminals updating central inventory counts or daily register summaries experienced prolonged row-level lock waits.
- **Deadlock Risks:** Inconsistent acquisition ordering across `orders` and `order_items` tables caused occasional transaction rollbacks.
- **Duplicate Transaction Risks:** Lack of client-side idempotency keys meant cashiers re-submitted pending orders when network latency spiked.
- **Isolation Level Verification:** Evaluated Read Committed vs Repeatable Read tradeoffs to balance dirty read prevention against serialization errors.

D. Verified Optimization Approaches — REMEDIATION BLUEPRINT

Targeted engineering remedies deployed or recommended by LOGIXOTECH:

- **Composite & Partial Indexing:** Created targeted B-tree indexes on `(outlet_id, created_at)` and partial indexes for open active orders.
- **Connection Pool Architecture:** Implemented transaction-mode connection pooling (PgBouncer), capping backend processes while serving hundreds of terminals.
- **Batch Insertions & CTEs:** Replaced multi-roundtrip row inserts with single atomic Common Table Expression (CTE) write queries.
- **Idempotent Order Ingestion:** Enforced unique client-generated UUID keys to guarantee zero duplicate charges during network reconnects.

Engineering Discipline: Distinguishing Symptoms from Root Causes

In complex enterprise systems, performance degradation following a feature release is rarely caused by a single bug. It is typically an amplification effect where unindexed queries increase database CPU, causing slower execution, which holds open database connections longer, rapidly exhausting connection pool limits and triggering timeout cascades across the client fleet.

6. MULTI-LOCATION SCALABILITY ARCHITECTURE: 350-LOCATION PRODUCTION FLEET

Production Workload Architecture Supporting Enterprise Multi-Outlet Deployment

To support the operational scale of Perera & Sons, the platform architecture accommodates an extensive network of **approximately 350 retail and food-service outlets**. This operating footprint provides the foundational architecture to maintain database stability, sub-second query latency, and reliable connection handling during peak daily meal periods.

PARAMETER	PRODUCTION OPERATING SCALE	ARCHITECTURAL IMPACT ON SHARED DATABASE
Business Locations	**350**	Multi-tenant data partitioning requirement; records scoped by `outlet_id`.
Dedicated POS Terminals	**350**	Continuous checkout writes, receipt printing, cash drawer reconciliation.
Staff Mobile Devices	**700**	Two mobile staff phones/tablets per outlet for table-side orders & inventory checks.
Total Active Devices	**1,050 Devices**	Simultaneous concurrent HTTP/WebSocket persistent connections to API layer.
Transaction Character	**Concurrent Sales & Orders**	Sub-second write requirement; high-frequency inserts into line-item tables.
Operational Model	**Centralized Cloud Services**	Shared PostgreSQL cluster hosting sales ledgers, catalogs, and reporting views.

MULTI-OUTLET CONCURRENCY DATA FLOW ARCHITECTURE

How 1,050 Distributed Client Devices Route into a Shared Cluster

LAYER 01: OUTLET FLEET

Distributed Edge Fleet

350 POS Terminals: Barcode scanning, receipt printing, payment gateway integration.

700 Staff Handhelds: Floor order taking, table status monitoring, inventory lookup.

→

LAYER 02: API & POOLER

Application & Pool Gateway

- **Load Balancer:** SSL termination & regional request routing.
- **API Services:** Stateless Next.js / Node.js worker microservices.
- **Connection Pooler:** PgBouncer multiplexing 1,050 client connections into a controlled backend pool.
- **Rate Limiter & Cache:** Redis-backed catalog caching.

→

LAYER 03: DATA CORE

PostgreSQL Database

Partitioned sales records, composite indexes, and strict ACID transaction boundaries.

Asynchronous read replicas dedicated to analytics and management reporting queries.

Production Concurrency Architecture

Across approximately 350 commercial outlets, the platform manages over 1,050 active devices simultaneously. The multi-tenant architecture scopes transactions cleanly by `outlet_id`, maintaining full ACID guarantees during continuous storefront ordering and central inventory synchronization.

7. PERFORMANCE BENCHMARK: BEFORE VS. AFTER OPTIMIZATION

Measured Production Results Following LOGIXOTECH Database Remediation

To demonstrate the quantifiable impact of database query optimization, connection pooling, and lock contention mitigation, the table below illustrates performance metrics before and after targeted engineering remediation under a peak checkout simulation:

PERFORMANCE METRIC	PRODUCTION BASELINE (PRE-FIX)	OPTIMIZED RESULT (POST-FIX)	MEASURED DELTA
API Response Time (p95)	**920 ms**	**420 ms**	**54.3% Latency Drop**
Database Query Response Time	**450 ms**	**120 ms**	**73.3% Latency Drop**
Database CPU Utilization	**85%**	**45%**	**47.1% Load Reduction**
Failed Request Rate	**5.0%**	**0.5%**	**90.0% Error Reduction**
Peak Database Connections	**450**	**250**	**44.4% Pool Efficiency**
Transaction Processing Error Rate	**2.0%**	**0.2%**	**90.0% Reliability Gain**

VISUAL METRIC IMPROVEMENT INDICATORS

Key Operational Threshold Reductions

API RESPONSE TIME (P95)
~~920 ms~~ **420 ms**
-54.3% Reduction

DATABASE QUERY LATENCY
~~450 ms~~ **120 ms**
-73.3% Latency Drop

DATABASE CPU UTILIZATION
~~85%~~ **45%**
-47.1% CPU Relief

FAILED REQUEST RATE
~~5.0%~~ **0.5%**
10x Error Reduction

PEAK DATABASE CONNECTIONS
~~450~~ **250**
-44.4% Connection Load

TRANSACTION ERROR RATE
~~2.0%~~ **0.2%**
99.8% Success Rate

Verified Production Impact
Through systematic slow query elimination, composite indexing, transaction-mode connection pooling (PgBouncer), and lock mitigation, LOGIXOTECH successfully reduced p95 API response times by 54.3% and slashed database query latency by 73.3%, permanently resolving cashier terminal freezes during peak customer traffic.

8. TECHNICAL OPTIMIZATION METHODOLOGY

Core Engineering Principles Applied to Database & API Layers

Query & Index Optimization

EXECUTION PLANS

- **EXPLAIN ANALYZE:** Evaluating costs, actual rows, and filter conditions to remove sequential table scans.
- **Composite B-Tree Indexes:** Aligning index definitions with query `WHERE`, `ORDER BY`, and join predicates.
- **Partial Indexing:** Indexing only active orders (e.g. `WHERE status = 'pending'`) to reduce index maintenance costs.

Connection Management

POOL ARCHITECTURE

- **PgBouncer Pooling:** Utilizing transaction-mode pooling to reuse backend server processes immediately after query completion.
- **Connection Limits:** Defining strict client connection ceilings to prevent thundering-herd database crashes.
- **Fast Session Release:** Eliminating long-lived idle-in-transaction connections across application workers.

Transaction Concurrency

LOCK MITIGATION

- **Atomic Processing:** Wrapping multi-item order checkouts in single atomic transactions.
- **Deterministic Locking:** Sorting resource updates in fixed order across threads to prevent cross-table deadlocks.
- **Non-Blocking Reads:** Leveraging Multi-Version Concurrency Control (MVCC) so queries never block writes.

9. REPEATABLE BENCHMARKING & LOAD-TESTING FRAMEWORK

Five-Stage Controlled Verification Lifecycle

LIFECYCLE STAGE	CORE OBJECTIVE & PROCEDURES	TELEMETRY & TOOLS	KEY DELIVERABLE
Stage 1: Baseline Audit	Inspect active system schema, query logs, API latency percentiles, active connections, and lock trees under normal traffic.	`pg_stat_statements`, buffer hit ratio, slow query logs.	Empirical Baseline Performance Report.
Stage 2: Synthetic Data	Generate representative multi-tenant dataset mirroring real retail outlets, catalog depths, and order histories without exposing customer PII.	Python synthetic data seeders, schema generators.	Sanitized Staging Database Cluster.
Stage 3: Load Simulation	Simulate hundreds of POS terminals and mobile devices executing concurrent checkout orders, menu fetches, and ticket submissions.	Automated **k6** / **Locust** distributed load test runners.	Repeatable Test Execution Harness.
Stage 4: Bottleneck Triage	Measure p50, p95, p99 latencies, requests per second (RPS), database CPU, lock wait queues, and transaction rollback percentages.	Grafana dashboards, PostgreSQL lock wait monitors.	Identified Bottleneck Root-Cause Register.
Stage 5: Remediation & Re-Test	Deploy indexes, connection poolers, and query rewrites; re-execute identical load workloads to prove quantifiable gains.	Automated comparative benchmarking suites.	Before-and-After Verified Audit Report.

10. DATABASE INTEGRITY & CONCURRENCY RELIABILITY

Architectural Mechanisms Safeguarding Financial & Inventory Consistency

Idempotent Transactions — ZERO DUPLICATES

Utilizing client-generated UUIDs and unique database constraints on orders to guarantee that transient cellular network reconnects never generate duplicate order charges.

Atomic Inventory Commits — ACID COMPLIANCE

Enforcing atomic updates on stock balances within the same database transaction as sales receipt generation, preventing out-of-stock overselling anomalies.

Lock Mitigation Protocols — HIGH CONCURRENCY

Implementing `FOR UPDATE SKIP LOCKED` and advisory locks for high-frequency queues, allowing concurrent workers to process orders without blocking the primary table.

11. DIRECT TECHNICAL RELEVANCE TO THE YOUZEN PLATFORM

Applying Proven POS & Booking Concurrency Engineering to Luca's Platform

The technical lessons derived from multi-outlet POS optimization directly map to the architectural demands of the **YouZen platform (supporting 100 restaurants and 100 hair salons on a shared PostgreSQL/Supabase database)**:

YOUZEN OPERATIONAL AREA	CONCURRENCY & DATABASE CHALLENGE	LOGIXOTECH ENGINEERING APPROACH
Restaurant Checkout Rush (100 POS + 200 phones)	Simultaneous meal-time checkouts, table bill splits, and kitchen ticket dispatch creating hot-row lock queues on `orders`.	Composite indexes on `(location_id, status)`, atomic CTE order ingestion, and PgBouncer connection multiplexing.
Hair Salon Bookings (Calendars & availability)	Multi-user race conditions where concurrent online users and phone desks attempt to book the exact same stylist time slot.	PostgreSQL exclusion constraints (`EXCLUDE USING gist`) and range types (`tsrange`) to physically eliminate double-bookings.
Supabase Realtime Traffic (Live kitchen & calendar sync)	Hundreds of mobile staff devices listening to Postgres WAL changes, triggering connection exhaustion and redundant reads.	Scoping Realtime channel subscriptions strictly by `location_id` and implementing client-side debounced invalidation.
Row-Level Security (RLS) (Tenant data protection)	Multi-tenant RLS policies on shared tables re-evaluating subqueries on every single scanned row, spiking query latency.	Refactoring RLS policies to use session-cached JWT claims and `STABLE` security-definer helper functions.

Critical Independence Statement: YouZen Requires Independent Staging Telemetry

LOGIXOTECH emphasizes that while our multi-outlet POS optimization experience provides deep architectural domain knowledge, **YouZen represents a distinct software platform that requires its own empirical baseline measurement and controlled staging tests.** Historical results from the Perera & Sons engagement are not proof of YouZen's specific performance, but demonstrate our team's systematic ability to isolate bottlenecks and optimize high-concurrency database platforms.

12. DEDICATED LOGIXOTECH TEAM RESPONSIBILITIES FOR YOUZEN

Explicit Accountability for Every Deliverable Requested by Luca

Viranga Lakshan — Technical Lead & Database Eng.

Primary Technical Owner

- Personally leads the PostgreSQL / Supabase technical audit and query profiling.
- Analyzes slow queries, lock contention, and RLS policy overhead.
- **Personally implements all agreed database fixes**, index migrations, and SQL rewrites.
- Validates database optimization results and provides architectural sign-off.

Janith Kavishka — Backend & Load-Testing Eng.

Primary Load-Testing Owner

- Leads load-test planning, scenario design, and scripting in **k6** / **Locust**.
- Prepares synthetic data generators for 100 restaurants and 100 salons.
- Executes independent and combined peak load tests on staging.
- Profiles API bottlenecks and produces before-and-after latency comparisons.

Lithumitha Ravikumar — Project Coordination & Client Comms

Primary Coordination Owner

- Manages project timelines, scope tracking, and milestone deliverables.
- Coordinates daily client communication and schedules technical meetings.
- Maintains technical runbooks, changelogs, and audit documentation.
- *Note: Role is project management and coordination, not primary database coding.*

13. HISTORICAL TEAM CONTRIBUTIONS & GOVERNANCE TRANSPARENCY

Distinguishing Past Project Involvements from Proposed Allocations

Historical Project Involvements: In previous technical optimization engagements, including the Perera & Sons investigation, LOGIXOTECH deployed specialized engineering personnel matching specific phase needs. Technical diagnosis, database indexing, and query tuning were directed by senior database architects, supported by systems testing engineers and engagement coordination managers.

Governance Statement: *Individual contributions should be confirmed against the project's actual delivery records. The team responsibilities described for YouZen represent the proposed allocation for the new engagement.*

14. CONFIDENTIALITY & DATA PROTECTION

Protecting Client Intellectual Property

This document has been prepared with strict adherence to client non-disclosure standards. All sensitive operational data, internal server IP addresses, credentials, private schema definitions, and customer identifiers have been fully anonymized. Reference to Perera & Sons is made exclusively within the permitted boundaries of technical case study review.

15. CONCLUSION & TECHNICAL COMMITMENT

Delivering High-Performance, Scalable Database Foundations

LOGIXOTECH approaches multi-tenant database optimization through empirical, measurable engineering: isolating root causes with rigorous telemetry, proving hypotheses through automated staging simulations, and implementing surgical database fixes that permanently protect transaction throughput. We are fully prepared to apply these proven principles to ensure the YouZen platform scales seamlessly across 100 restaurants and 100 hair salons.

16. CLIENT REFERENCE & PROJECT VERIFICATION

Formal Verification Protocols for International Prospective Clients

Project Reference Profile – Perera & Sons POS Optimization

ENTERPRISE PROFILE

Client Organization: Perera & Sons
Service Provider: LOGIXOTECH Technical Team
Industry Sector: Retail and Food & Beverage (F&B)
Project Scale: ~350 Commercial Outlets Operating Concurrently

Project Scope: Performance investigation and optimization of an existing multi-outlet POS platform following the introduction of additional application features.

Engagement Summary

SCOPE VERIFIED

LOGIXOTECH was engaged to investigate operational performance and response-time degradation affecting the existing centralized POS platform. The technical engagement focused specifically on database query execution analysis, index architecture, connection pool management, concurrent transaction handling, API-level profiling, and multi-location system scalability.

Client Reference Contact Information

ENTERPRISE PROTOCOL

Direct contact facilitated via authorized introduction bridge:

- **Reference Contact:** Available upon formal introduction via LOGIXOTECH management
- **Official Position:** Head of Information Technology / Retail Systems
- **Organization:** Perera & Sons
- **Business Email & Phone:** Facilitated via authorized introductory bridge
- **Permission to Contact:** Confirmed for prospective client evaluations

Project Verification Framework

VALIDATION SCOPE

Subject to client authorization, LOGIXOTECH can facilitate a reference discussion to formally verify:

- **1. Scope of Engagement:** Nature and technical boundaries of the engagement.
- **2. LOGIXOTECH Involvement:** Direct leadership of the POS performance investigation.
- **3. Team Responsibilities:** Database and testing roles performed by the team.
- **4. Scale Supported:** Over 350 commercial business locations and 1,050 active devices.
- **5. Remediation Delivered:** Performance investigation and optimization activities undertaken.

Confidentiality Statement & Verification Governance

Client contact details and operational project information are protected under standard corporate confidentiality standards. LOGIXOTECH coordinates direct reference calls and technical verification discussions with Perera & Sons leadership for prospective enterprise clients to review the engagement scope, engineering responsibilities performed, and delivered database optimizations.